

Shrinking an ECDSA Verifier to 933 Bytes

A beginner’s guide to the tricks that worked

March 2026

Abstract

This report walks through how an ECDSA/P-256 signature verifier was shrunk from **3076 B** of portable C down to **933 B** of x86-64 machine code — a factor of $3.3\times$. The target context is a boot ROM: the code must fit in a tiny mask-programmed memory, has no libc, and verifies exactly one signature before jumping to firmware. Every byte in a boot ROM is paid for in silicon.

We focus on *why* each trick works, not just *that* it works. No prior exposure to elliptic curves is assumed; we introduce just enough of the mathematics to make sense of the size wins. The full source, test suite, and commit history live in `tv_ecdsa_tiny.S` and the surrounding repository.

Contents

1	The problem	2
1.1	The ground rules	2
1.2	What ECDSA verification actually computes	3
2	The journey at a glance	3
3	Bytecode: call sites are expensive, interpreters are cheap	3
3.1	The problem with calling functions	4
3.2	The fix: a tiny virtual machine	4
3.3	Why 2 bytes, why nibbles	5
4	Dropping Montgomery: the $q = t[\text{top}]$ reduce	5
4.1	What Montgomery multiplication costs you	5
4.2	The special shape of the P-256 moduli	5
4.3	Why that shape makes reduction trivial	5
5	The projective final check: deleting an inversion	6
5.1	Where the mod- p inversion was hiding	6
5.2	Clearing denominators	7
5.3	Folding the OR into a multiply	7
6	bt on n directly: no exponent buffer	7
6.1	The cost of computing $n - 2$	7
6.2	The bit-level observation	7
6.3	The broader lesson	8

7	RCB complete addition: one formula for everything	8
7.1	Why point addition normally has three cases	8
7.2	The complete formula	8
7.3	The demolition	9
7.4	The fine print	9
8	Layout is free, setup is not	9
8.1	Shamir’s trick and its setup cost	9
8.2	Making the source contiguous	10
9	Building constants at runtime	10
9.1	A constant you can compute in the same number of bytes	10
9.2	The knock-on win	11
10	The EFD reschedule: reorder for displacement reach	11
11	The micro-grind: x86 encoding folklore	11
11.1	push; pop is a 2-byte mov	11
11.2	Short jumps versus near jumps	12
11.3	Fall-through instead of jump	12
11.4	rep movsq is a free memcpy	12
11.5	Microcode is slow but short	12
11.6	The rcx = 0 flow	12
12	Things that didn’t work	12
13	General principles	13

1 The problem

A boot ROM is a small block of read-only memory burned into a chip at the factory. When the chip powers on, it runs this code first — before any firmware, before any operating system. If the boot ROM verifies the firmware’s signature before handing over control, the chip has a *hardware root of trust*: even an attacker who can overwrite flash cannot run unsigned code.

The catch is size. Mask ROM cells are large compared to flash, every chip on the wafer carries the same bits, and respinning silicon to fix a bug costs months and seven figures. Manufacturers budget boot ROMs in *kilobytes*. Your verifier shares that budget with the flash driver, the clock setup, and whatever else the chip needs to come alive. If you spend 3 KB on signature verification, something else gets cut.

So the game is: **implement ECDSA signature verification over curve P-256, as small as you can make it, without cutting corners on correctness.**

1.1 The ground rules

- **Count everything.** Size is measured on the *object file* — text plus read-only data. If you call `memcpy`, `memcpy` counts. The target here has zero undefined symbols: no `libc`, no compiler runtime, nothing.
- **Correctness is not negotiable.** Every build must pass 607 test vectors: 33 hand-picked edge cases (zero components, out-of-range values, the point at infinity, signature malleability) plus 574 from Google’s Wycheproof suite [5]. A verifier that’s ten bytes smaller but accepts a forged signature is worth exactly nothing.

- **Speed is secondary but not irrelevant.** A boot ROM runs once per power-on, so milliseconds are fine. But there’s a *Pareto frontier*: if build A is both smaller *and* faster than build B, build B is dead. We track both axes.
- **Constant time is not required.** Signature *verification* takes only public inputs — the signature, the public key, the message hash. Nothing secret. Data-dependent branches and variable-time reductions, which would be disqualifying in a *signer*, are fair game here. Several of the tricks below exploit this freedom.

1.2 What ECDSA verification actually computes

You do not need to know why ECDSA is secure to follow this report. You only need to know what the verifier *does*, because that determines where the bytes go.

An ECDSA signature over P-256 is a pair of integers (r, s) , each 256 bits wide. The public key is a point Q on an elliptic curve. Verification, per FIPS 186-5 [1], computes:

$$\begin{aligned}
 w &= s^{-1} \bmod n && \text{(one modular inversion)} \\
 u_1 &= e \cdot w \bmod n && (e = \text{the message hash}) \\
 u_2 &= r \cdot w \bmod n \\
 (x, y) &= u_1 \cdot G + u_2 \cdot Q && \text{(two scalar multiplications, one point add)} \\
 &\text{accept iff } x \equiv r \pmod{n}
 \end{aligned}$$

Here G is a fixed “generator” point baked into the standard, n is the order of G (a 256-bit prime), and p is the field modulus (another 256-bit prime). All the elliptic-curve arithmetic happens modulo p ; the scalars u_1, u_2, w live modulo n .

The heavy lifting is in $u_1 \cdot G + u_2 \cdot Q$. Computing $k \cdot P$ means “add the point P to itself k times” — but since k is a 256-bit number, you do this with a square-and-multiply style loop: 256 iterations of *point doubling* (compute $2P$ from P) interleaved with *point addition* (compute $P + Q$ from P and Q) wherever the scalar has a 1 bit.

Each point doubling or point addition is in turn a fixed sequence of field operations: multiplications, squarings, additions, subtractions, all modulo p . A P-256 point addition is roughly a dozen field multiplications plus change. One standard trick: instead of working with (x, y) pairs directly, the loop carries an extra coordinate Z and represents the running point as (X, Y, Z) with $x = X/Z$. Divisions are expensive (see §5); this *projective* representation defers them all to one division at the very end.

So the byte budget is roughly: one field multiplier (mod p and mod n), one point-add and one point-double formula that call it, one scalar-multiply loop that calls *them*, one inversion, and the top-level glue that decodes the inputs and checks the result. Constants for p, n, G , and the curve coefficient b take another 128–160 bytes.

2 The journey at a glance

Table 1 gives the headline numbers. The rest of this document goes through each row in enough detail that you could, in principle, rediscover the trick yourself.

3 Bytecode: call sites are expensive, interpreters are cheap

From **2873 B** to **1712 B**.

Size	Δ	What changed
3076 B		Portable C, gcc -Os, 32-bit limbs
2873 B		First hand-written x86-64 assembly
1712 B	-1161	§3: Bytecode interpreter
1427 B	-285	fast.S: Montgomery + mulx + Shamir’s trick
1397 B	-30	Micro-grind (register allocation, fall-through)
1300 B	-97	§4: Drop Montgomery — the $q = t[\text{top}]$ reduce
1195 B	-105	§5: Projective final check — no mod- p inversion
1105 B	-90	§6: bt on n directly — no exponent buffer
1012 B	-93	§7: RCB complete addition — one formula, no branches
985 B	-27	§8: Addend slot shift — layout is free, setup is not
942 B	-43	§9: Build p at runtime + fe_iszero inlines
933 B	-9	§10: EFD reschedule — reorder for disp8 reach

Table 1: Major milestones. Sizes between rows are connected by dozens of smaller wins; see §11 for the general flavour. The rule marks where tiny.S forked from fast.S: rows above it optimised for speed first; rows below trade speed for size.

3.1 The problem with calling functions

The first hand-written assembly version looked the way you’d expect: a function for field-multiply, a function for field-add, a function for field-subtract, and then pt_dbl and pt_add calling them in sequence.

On x86-64, calling a three-argument function costs about 15 bytes:

```

lea    rdi, [r14 + 32*DST]    ; 4 bytes (or 7 if disp > 127)
lea    rsi, [r14 + 32*SRC1]   ; 4 bytes
lea    rdx, [r14 + 32*SRC2]   ; 4 bytes
call   fe_mul                 ; 5 bytes

```

Point doubling is about 8 field operations; point addition is about 12. The verifier’s final-check sequence is another 10 or so. That’s roughly 60 call sites $\times$ 15 bytes each: **900 bytes spent on nothing but “load three pointers and jump.”**

3.2 The fix: a tiny virtual machine

Notice that every one of those call sites has the same *shape*: one destination, two sources, one operation. And all the operands live in a contiguous block of 32-byte field elements on the stack. So instead of emitting 15 bytes of setup per call, we encode each call as **2 bytes of data** and write a small loop that decodes and dispatches:

```

; Each instruction: byte0 = (s2<<4)/op, byte1 = (dst<<4)/s1
; With 16 slots and 16 ops, everything fits in nibbles.
bc_run:
    lodsw                ; fetch 2 bytes of bytecode into ax
    ; ... decode nibbles into rdi/rsi/rdx (about 30 bytes) ...
    movzx ecx, byte [r12 + rax] ; jump-table lookup (op index -> handler offset)
    add     rcx, r12
    call    rcx            ; dispatch
    jmp     bc_run

```

The dispatch loop is about 60 bytes. Each handler (multiply, add, subtract, ...) is another 10–40 bytes — but you already had those as functions. The bytecode streams

themselves are $60 \times 2 = 120$ bytes.

Net effect: ~ 900 bytes of call sites becomes ~ 180 bytes of interpreter plus bytecode. The `pt_dbl` function literally *disappears* as native code — it is now 25 bytes of data in `.rodata`.

3.3 Why 2 bytes, why nibbles

You could encode each op in 3 bytes (one per operand) and have room for 256 slots. But you don't *have* 256 slots — point addition uses perhaps a dozen temporaries. Four bits per operand, packed two to a byte, is the right granularity. And four bits of opcode gives you 16 operations — the final build uses ten.

There's a subtle win hiding here: **the interpreter doesn't care which *specific* 16 slots you use**. You can reassign slots to make other code shorter. We will exploit this ruthlessly later (§8).

4 Dropping Montgomery: the $q = t[\text{top}]$ reduce

From **1397 B** to **1300 B**. Cornerstone of the size-optimised build.

4.1 What Montgomery multiplication costs you

The fast implementation used *Montgomery form* [4]: numbers are stored as $a \cdot R \bmod m$ where $R = 2^{256}$. This makes the reduce-after-multiply step very fast — but it carries hidden fixed costs:

- A precomputed constant $m_0^{-1} = -m^{-1} \bmod 2^{64}$ for each modulus (p and n): **16 B** of data plus the code to pass it into the multiplier on every call.
- A precomputed constant $R^2 \bmod p$ to convert inputs *into* Montgomery form: **32 B** of data plus a TOMONT bytecode op.
- Conversion *out* of Montgomery form at the end: more bytecode ops.

None of these are big individually, but they add up to about 60–80 bytes — and for size, we'd rather have a slower multiplier with no baggage.

4.2 The special shape of the P-256 moduli

Here is where we get lucky. Write out p and n in hex and stare at the top word:

```
p = ffffffff 00000001 00000000 00000000 00000000 ffffffff ffffffff ffffffff
n = ffffffff 00000000 ffffffff ffffffff bce6faad a7179e84 f3b9cac2 fc632551
```

Both moduli have a top 32-bit word of 0xFFFFFFFF. And both satisfy $2^{256} - m < 2^{224}$ — that is, both are “just barely less than” 2^{256} .

4.3 Why that shape makes reduction trivial

After a schoolbook multiply, you have a 512-bit product t and you want $t \bmod m$. The textbook approach is trial division: estimate the quotient, multiply, subtract, correct. Quotient estimation is where the complexity lives.

But if the top word of m is 0xFFFFFFFF, quotient estimation becomes **free**. Split t into 32-bit words $t[0..15]$ and slide a 9-word window from the top down. At each position j :

1. Let $q = t[j+8]$ — just *read* the top word of the window. No division.

2. Subtract $q \cdot m$ from the window.

Why is $q = t[j+8]$ a good quotient estimate? Because m 's top word is $2^{32} - 1$, the true quotient $\lfloor t_{\text{window}}/m \rfloor$ is within 1 of $\lfloor t_{\text{window}}/2^{256} \rfloor = t[j+8]$.

Concretely: write the window as $W = q \cdot 2^{256} + L$ with $0 \leq L < 2^{256}$. Then

$$W - q \cdot m = q(2^{256} - m) + L < 2^{32} \cdot 2^{224} + 2^{256} = 2 \cdot 2^{256},$$

so after one subtraction the window is at most two moduli wide, and its new top word — $\lfloor (W - qm)/2^{256} \rfloor$ — is either 0 or 1. If it's 1, one more pass brings it to 0. Two passes per position, worst case.

```
; Reduce: slide a 9-dword window from j=8 down to j=0.
; At each j: q = t[j+8]; t[j..j+8] -= q*m.
; The SAME 26-byte primitive handles both schoolbook multiply
; AND this reduce --- they're both "mul ebx; accumulate into [rdi]".
.Lreduce:
    mov     ebx, [rdi + 32]    ; q = top dword of current window
    ; ... call the shared mul-accumulate primitive ...
    sub     rdi, 4             ; slide window down one dword
    dec     ecx
    jnz     .Lreduce
```

The payoff: no m_0^{-1} , no R^2 , no conversion ops. Field elements are stored in plain form the whole time. The reduce code is *shorter* than Montgomery's, and one 26-byte inner primitive serves double duty for multiply and reduce.

The trade-off is speed: this runs at 32-bit granularity (8×8 word multiplies instead of 4×4), so it's about $2 \times$ slower. For a boot ROM, we'll take that trade every time.

A subtlety worth noting. The $q = t[\text{top}]$ trick requires the top *32-bit* word to be all-ones. The top *64-bit* word of p is `0xFFFFFFFF00000001`, which is *not* all-ones. So even after we later upgraded the schoolbook multiply to 64-bit limbs (**+14 B** for about a $2 \times$ speedup), the reduce stayed at 32-bit granularity. This is why `tv_ecdsa_tiny.S` has a slightly odd split: 64-bit multiply, 32-bit reduce.

5 The projective final check: deleting an inversion

*From **1216 B** to **1195 B**. One of those rare wins that's both smaller and faster.*

5.1 Where the mod- p inversion was hiding

The ECDSA check (last line of §1.2) is “accept if $x \equiv r \pmod{n}$,” where x is the affine x -coordinate of the computed point. But the scalar multiplication loop doesn't produce affine coordinates — it produces *projective* coordinates (X, Y, Z) , where the affine x is recovered as $x = X/Z$ (for homogeneous projective) or $x = X/Z^2$ (for Jacobian).

“Divide by Z ” in a finite field means “multiply by Z^{-1} .” For prime p , Fermat's little theorem gives $Z^{p-1} \equiv 1 \pmod{p}$, so $Z^{-1} \equiv Z^{p-2}$: inversion is exponentiation. That exponentiation is a 256-iteration square-and-multiply loop — another ~ 380 field multiplications on top of everything else.

The straightforward verifier does this inversion, recovers x , and compares it to r . That inversion call, its argument setup, and the final compare-and-reduce-mod- n block cost about 45 bytes.

5.2 Clearing denominators

Here's the algebra trick. Instead of computing $x = X/Z$ and checking $x \stackrel{?}{=} r$, **clear the denominator**: check $X \stackrel{?}{\equiv} r \cdot Z \pmod{p}$. No inversion needed.

There's one wrinkle. The ECDSA check is modulo n , but X and Z live modulo p , and $p \neq n$. Since $0 \leq x < p < 2n$, there are at most two integers congruent to $r \pmod{n}$ in that range: r itself, and $r + n$ (the latter only when $r + n < p$). So the full check becomes:

$$X \equiv r \cdot Z \pmod{p} \quad \text{OR} \quad X \equiv (r + n) \cdot Z \pmod{p}.$$

When $r \geq p - n$ — a roughly 2^{-128} sliver of the r range — the sum $r + n$ wraps around mod p and the second disjunct tests a spurious value. That's harmless: forcing the scalar multiplication to land on a *specific* spurious point is as hard as the discrete log problem the whole scheme rests on.

5.3 Folding the or into a multiply

Checking an OR in assembly usually means a branch, and branches are byte-expensive. But we're working modulo a prime p , and in a field, **a product is zero iff one of its factors is zero**. So let

$$d_1 = X - r \cdot Z, \quad d_2 = X - (r + n) \cdot Z,$$

and check whether $d_1 \cdot d_2 \equiv 0 \pmod{p}$. One multiply, one zero-check, no branches.

The whole thing — compute $r \cdot Z$, compute d_1 , compute d_2 , multiply, check-zero — is 7 field operations, which is 14 bytes of bytecode. The 45 bytes of native inversion-and-compare are gone, replaced by 14 bytes of data.

And as a bonus, we no longer call the inverter with modulus p at all. The inverter now only ever runs mod n (for s^{-1}). That removes an INV_P handler from the bytecode jump table and lets us hardcode n into the inverter, saving another handful of bytes (§6).

6 bt on n directly: no exponent buffer

From [1124 B](#) to [1105 B](#).

6.1 The cost of computing $n - 2$

Fermat's little theorem gives $s^{-1} \equiv s^{n-2} \pmod{n}$. The exponentiation is a 256-step square-and-multiply loop that reads one bit of the exponent $n - 2$ per iteration: square always, multiply if the bit is 1.

The exponent is $n - 2$. Previously we stored n as a constant, and at runtime: copy n to a 32-byte stack buffer, subtract 2 from the buffer, then have the loop read bits from the buffer with the `bt` instruction.

That's an `enter` to allocate the buffer, a `rep movsq` to copy, a `sub` to decrement, and a `leave` to clean up: 33 bytes of machinery just to turn n into $n - 2$.

6.2 The bit-level observation

Write out the low byte of n and $n - 2$ in binary:

$$\begin{aligned} n \bmod 256 &= 0x51 = 0101\ 0001 \\ (n - 2) \bmod 256 &= 0x4F = 0100\ 1111 \\ \text{XOR} &= 0x1E = 0001\ 1110 \end{aligned}$$

Subtracting 2 borrows from bit 1. That bit is 0 in n , so the borrow propagates: through bit 2 (also 0), through bit 3 (also 0), until it reaches bit 4 — which is 1 and absorbs it. **Bits 5 through 255 of $n - 2$ are identical to bits 5 through 255 of n .**

So: don't compute $n - 2$ at all. Read bits directly from the n constant that's already sitting in `.rodata`. For bits 5 and up, n gives the right answer. For bits 0–4, patch in a 7-byte special case:

```

    cmp    ebx, 4
    jb     .Lmul      ; bits 0-3: n-2 has 1111 here, force multiply
    je     .Lnext     ; bit 4:    n-2 has 0, n has 1, force skip
    bt     [r12 + oN], ebx ; bits 5+: n and n-2 agree, read n directly
    jnc    .Lnext
.Lmul:
    ; ... multiply step ...

```

The 33-byte buffer machinery becomes 7 bytes of branching. Six bytes of the original machinery survive (the init copy still has to run), and the `bt` operand grows by one byte (it now reaches into `.rodata` instead of `[rsp]`). Net: $-33 + 7 + 6 + 1 = -19$ B.

6.3 The broader lesson

This trick is *ludicrously specific* to the bit pattern of the P-256 group order. It would not work for most other primes. But that's exactly the point: when optimising for size, there is no “generic” — you stare at the *actual* constants and ask what accidents of their structure you can exploit.

7 RCB complete addition: one formula for everything

From **1071 B** to **1012 B**. The biggest single-commit win after the bytecode interpreter.

7.1 Why point addition normally has three cases

The classical formulas for adding two elliptic curve points $P + Q$ have a dirty secret: they divide by $x_P - x_Q$. When $P = Q$ (doubling), that's 0/0. When $P = -Q$ (the result is the point at infinity $\mathcal{O}$), same problem. And if either input is already $\mathcal{O}$, the formulas don't even apply — $\mathcal{O}$ has no coordinates.

So a traditional `pt_add_acc` routine starts with a three-way branch:

```

if (acc.z == 0)      { acc = Q; return; }      // acc was infinity
// ... run first half of add formula to compute H = x_Q - x_P ...
if (H == 0) {
    if (Rr == 0)      { pt_dbl(acc); return; } // P == Q, use doubling
    else              { acc = infinity; return; } // P == -Q
}
// ... finish the addition formula ...

```

In our bytecode world this was 62 bytes of native branching, plus *three* separate bytecode streams: `bc_dbl` (50 B), `bc_add1` (24 B, the pre-check half), and `bc_add2` (28 B, the post-check half). Total: 164 bytes.

7.2 The complete formula

In 2015, Renes, Costello, and Batina [2] published a point addition formula that **has no exceptional cases**. One sequence of 43 field operations gives the correct answer for *every*

pair of inputs: distinct points, equal points, inverse points, and the point at infinity. No branches. No special cases. It just works.

The formula uses *homogeneous* projective coordinates: the affine point is recovered as $(X/Z, Y/Z)$. Two triples name the same point whenever they differ by a common nonzero scale factor: $(X : Y : Z)$ and $(\lambda X : \lambda Y : \lambda Z)$ are the same thing, which is what the colons signal. Infinity is the class with $Z = 0$, which on this curve forces $X = 0$ too: any $(0 : Y : 0)$ with $Y \neq 0$ will do. Feeding the formula $(0 : 1 : 0) + Q$ really does produce Q , and $(0 : 1 : 0) + (0 : 1 : 0)$ really does produce $(0 : 1 : 0)$ — the edge cases are handled by the algebra, not by code.

7.3 The demolition

With RCB, `pt_add_acc` becomes:

```
pt_add_acc:                                ; 11 bytes. Was 62.
; Addend already in slots 5,6,7. Just dispatch bc_rcb.
xor     edi, edi                            ; bytecode offset 0 = bc_rcb
jmp     bcrun_off
```

Doubling is not a separate operation any more. To double the accumulator, copy it into the addend slots and call the *same* routine — $P + P = 2P$, and RCB handles that case like any other.

The three bytecode streams (**102 B** total) collapse into one 43-operation stream (**88 B**). The 62-byte branch tree becomes an 11-byte dispatch. Setup bookkeeping for the new Z coordinate eats **+6 B** of that back; net **−59 B**, and a whole category of bugs (forget to handle one edge case, accept a forged signature) structurally eliminated.

7.4 The fine print

RCB does more arithmetic than the classical “generic add” — 43 operations versus about 16. So each addition is about $2.5\times$ slower. For a boot ROM, we shrug. For a speed-optimised build, you’d keep the branchy version.

One invariant turned out to be load-bearing: the RCB formula never *writes* to the slots holding Q . It reads them, but the outputs go elsewhere. This means the scalar-multiply loop doesn’t need to save and restore Q across each add — Q just sits there, undisturbed, for all 256 iterations. That invariant saved another dozen bytes of copy-in-copy-out in the loop.

8 Layout is free, setup is not

From **1001 B** to **985 B**.

The bytecode interpreter addresses 16 slots by a 4-bit index. Which slot is “slot 5” is completely arbitrary — it’s a nibble in a data table. Renumbering slots costs **zero bytes** in the bytecode. But slot numbers show up in *native* code too, as memory displacements — and those are not free.

8.1 Shamir’s trick and its setup cost

Shamir’s trick [6, §14.88] (due originally to Straus [7]) computes $u_1G + u_2Q$ with a single 256-iteration loop instead of two: at each bit position, double once, then conditionally add G , conditionally add Q . It halves the number of doublings.

The trick needs backup copies of both G and Q that survive the whole loop, because the “working” addend slots get overwritten. After the RCB switch, each point is 3 slots (X, Y, Z). So the setup is: copy G (3 slots) and Q (3 slots) to backup slots 16–21.

The naive setup was three separate copies: G ’s X, Y from one place, a borrowed $Z = 1$ from another, Q from a third. About 28 bytes of `lea/rep movsq` triples.

8.2 Making the source contiguous

What if the six source slots were *already contiguous* before the copy? Then the setup is one `rep movsq` with count 24. Ten bytes instead of 28.

To make this happen: (a) arrange for the early validation bytecode to leave slots 2–7 holding exactly $G_x, G_y, 1, Q_x, Q_y, 1$ in that order; (b) shift the RCB addend from slots 4,5,6 to slots 5,6,7 so it lines up.

Part (a) costs 2 extra bytes of bytecode (one more “copy 1 here” op). Part (b) is a **pure nibble permutation**: swap some 4-bit fields in `bc_rcb`, zero-byte cost. Net: **−16 B**.

The general principle: data layout changes inside the bytecode are free; exploit that freedom to make the native code shorter.

9 Building constants at runtime

From **948 B** to **942 B** — but the direct saving is zero. The entire win is a knock-on effect.

9.1 A constant you can compute in the same number of bytes

The field prime p for P-256 is

$$p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1.$$

As 32 bytes of `.rodata` it’s **32 B**. But look at the hex:

```
ffffffff 00000001 00000000 00000000 00000000 ffffffff ffffffff ffffffff
```

Seven of the eight 32-bit words are either `0x00000000` or `0xffffffff` — only `0x00000001` is neither. Writing this out one dword at a time with `stosd` (which stores `eax` and auto-advances `rdi`) takes **17 B**:

```
or    eax, -1      ; eax = FFFFFFFF    (3 B)
stosd                ; dword 0          (1 B each; stosd = AB)
stosd                ; dword 1
stosd                ; dword 2
xor    eax, eax     ; eax = 0           (2 B)
stosd                ; dword 3
stosd                ; dword 4
stosd                ; dword 5
inc    eax          ; eax = 1           (2 B)
stosd                ; dword 6
neg    eax          ; eax = FFFFFFFF    (2 B)
stosd                ; dword 7
```

That looks like a **−15 B** win — but the constant didn’t sit in a vacuum. Deleting it means the places that *used* it need new plumbing: a pointer register to hold the built-at-runtime address, a new bytecode op so validation can check coordinates against it, save/restore of the new register in the prologue. About **+15 B** of overhead, netting out to roughly zero.

So why bother? Because **where the 32 bytes disappeared from matters more than how many were saved**.

9.2 The knock-on win

The bytecode jump table stores handler addresses as *8-bit* offsets (one byte per handler). This means every handler must be within 255 bytes of the table base. The largest handler (`fe_inv_m`) was sitting at offset 247 — only 8 bytes of headroom before the whole scheme breaks and every table entry doubles in size.

The `cP` constant lived between the jump table and the handlers. Deleting it from `.rodata` moved every handler 32 bytes closer to the table. Suddenly there’s 36 bytes of headroom — enough to inline a small `fe_iszero` helper into its only caller, for **−6 B**. That is the entire saving in the section header.

(We had already done the related trick for the curve coefficient b , computed *in bytecode* from the curve equation itself: since G is on the curve, $b = G_y^2 - G_x^3 + 3G_x \pmod{p}$. Four field ops, 8 bytes of bytecode, replacing **32 B** of constant — a direct saving this time, because bytecode costs no plumbing.)

Size optimisations interact: freeing space in one region can unblock a seemingly unrelated win elsewhere. Keep a list of “things I’d do if only I had 10 more bytes of headroom at X .”

10 The EFD reschedule: reorder for displacement reach

*From **935 B** to **933 B**. The last two bytes.*

x86-64 memory operands with a small displacement ($-128 \leq d \leq 127$) encode in one byte; larger displacements take four. One `lea rdi, [r14+160]` in the hot copy path was paying the three-byte tax because slot 5 is $5 \times 32 = 160$ bytes from the slot base, and $160 > 127$.

If we had a register that already pointed near slot 5, we could use a short displacement from that register. And `r8` was already holding `&cP` — but `cP` was at slot 22, way too far.

So: move cP. If `cP` lived at slot 8 instead ($8 \times 32 = 256$ from base), then slot 5 would be at `r8 - 96`, which fits in a signed byte. But slot 8 was being used as scratch by the RCB formula.

The Explicit-Formulas Database [3] lists the RCB operations in a particular order, but the order is not sacred — only the *data dependencies* are. By hoisting two independent steps earlier in the sequence, Z_1 dies sooner, slot 2 becomes reusable as a sixth temporary, and slot 8 is *never written* by `bc_rcb`. It can hold `cP` for the entire scalar multiplication.

Shuffling bytecode nibbles is free. The displacement shrinks from **+160** (4 bytes) to **−96** (1 byte). Minus one byte for a setup adjustment elsewhere, net **−2 B**.

Two bytes is small, but it illustrates how far the rabbit hole goes: *instruction scheduling in a data table to change register allocation to shrink an addressing mode in unrelated native code*.

11 The micro-grind: x86 encoding folklore

Between the big structural wins, hundreds of bytes came out one or two at a time. These don’t make for a good story individually, but the patterns repeat.

11.1 push; pop is a 2-byte mov

`mov rax, rbx` is 3 bytes. `push rbx; pop rax` is 2 bytes (for registers that don’t need a REX prefix). It clobbers 8 bytes of stack below `rsp`, which is fine if you’re not using the red zone for anything else.

This pattern appears about a dozen times in the final code. It saved roughly **−10 B** total.

11.2 Short jumps versus near jumps

A jump to a target within ± 127 bytes encodes as 2 bytes (`EB xx`). Beyond that, it's 5 bytes (`E9 xx xx xx xx`). Every function-layout decision — what order the handlers go in, whether a helper sits before or after its caller — affects which jumps are short.

Several commits in the history are just “swap these two blocks so this `jmp` goes from 5 bytes to 2.” Three bytes per swap, maybe five such swaps over the project: **-15 B**.

11.3 Fall-through instead of jump

Even better than a short jump is no jump. If function A always ends by calling function B, and nothing else sits between them, delete the `call` from A and let execution fall through. `Fadd` falls through into the conditional-subtract epilogue; `Nmul` overwrites the modulus pointer and falls through into `Fmul`. Each fall-through is **-2 B** to **-5 B**; four or five over the project comes to roughly **-15 B**.

11.4 `rep movsq` is a free `memcpy`

`rep movsq` is 3 bytes and copies `rcx` quadwords from `[rsi]` to `[rdi]`. If you can get `rsi`, `rdi`, and `rcx` into the right state as a side effect of surrounding code — `lodsb` leaves `rsi` advanced, `stosq` leaves `rdi` advanced, loop counters naturally end at zero — then a block copy is nearly free.

The final code has about eight `rep movsq` instructions and not a single explicit copy loop. The art is in arranging the dataflow so the registers are already right.

11.5 Microcode is slow but short

The `loop` instruction (decrement `rcx` and jump if nonzero) is 2 bytes. The equivalent `dec rcx; jnz` is 5 bytes. But `loop` is *microcoded* on modern x86 and costs several extra cycles per iteration — Agner Fog's tables [8] put it at 6 on Skylake, worse on AMD Zen.

If the loop runs ten times, nobody cares. If it's the inner loop of the field multiplier and runs 950,000 times per signature, that's millions of wasted cycles. The `-DSMALL_MUL8` build takes the size win; the default build spends **+14 B** to run at full speed. Both are on the Pareto frontier.

11.6 The `rcx = 0` flow

`mov cl, 8` is 2 bytes and sets the low 8 bits of `rcx`. If the upper 56 bits happen to already be zero — because a previous `rep` instruction counted down to zero, or a previous `loop` exited — you just saved 3 bytes over `mov ecx, 8` (5 bytes) or 1 byte over `push 8; pop rcx` (3 bytes).

This is a *global* invariant: “`rcx` is zero at these twelve points in the control flow, and the code downstream relies on it.” It's fragile — reordering two blocks can silently break it — and it saved perhaps **-8 B** across the whole binary. The source has comments at every point where the invariant is consumed.

12 Things that didn't work

Not every idea pans out. A few instructive failures:

Merging schoolbook and reduce into one loop.

The multiply loop and the reduce loop have similar shapes — walk an array, multiply by

something, accumulate. Merging them would share the loop machinery. It also overflows: the accumulated carry during the multiply phase can exceed what the reduce phase expects, and you get wrong answers on about one input in 2^{30} . Reverted after a Wycheproof failure.

Zero-skip in the scalar loop.

Half the scalar bits are zero, so skipping the add on zero bits should be a speedup. It was 40% *slower*: the branch mispredicts roughly half the time (the bits are effectively random) and the penalty swamps the saved work. `perf stat` showing branch-miss rate was the diagnostic that killed this idea.

Solinas reduction for both moduli.

The prime p has a beautiful sparse form ($2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$) that admits a reduction with no multiplies at all, just shifts and adds. But n doesn't have that form, so you need *two* reducers instead of one. For a speed build that's worth it (`-DSOLINAS_P` exists); for the size floor, one generic reducer serving both moduli is smaller.

13 General principles

If you're trying to shrink your own binary, here's what transfers:

1. **Count call sites, not functions.** A function called once is as expensive as its call site plus its body. A function called sixty times is sixty call sites plus one body. The interpreter trick (§3) wins because it collapses sixty 15-byte call sites into sixty 2-byte data entries.
2. **Algebra beats branching.** The projective check (§5) and the RCB formula (§7) both replace control flow with arithmetic. Arithmetic is data; branches are code. Data is cheaper.
3. **Your constants are not random.** The `bt-on- $n$` trick (§6), the $q = t[\text{top}]$ reduce (§4), and building p at runtime (§9) all exploit specific bit patterns in the P-256 constants. Stare at your constants in hex. Stare at them in binary. There is structure there.
4. **Layout is a degree of freedom.** Slot numbering (§8), function order (short jumps), constant order (block copies), rodata-before-text (push imm8) — none of these affect semantics, all of them affect size. Change them last, after the architecture settles, because they're brittle.
5. **Track both axes.** It's easy to micro-optimize yourself into a corner where you've saved 5 bytes and lost $10\times$ speed. Plot size against cycles for every checkpoint; if your new point is dominated by an old one on both axes, you've wasted your time.
6. **Tests before bench.** Every single one of the ~ 130 commits in this project passed 607/607 test vectors before being committed. Several ideas were killed by tests (see above). A smaller verifier that accepts forgeries is negative progress.

References

- [1] National Institute of Standards and Technology. *Digital Signature Standard (DSS)*. FIPS PUB 186-5, February 2023. <https://doi.org/10.6028/NIST.FIPS.186-5>
- [2] Joost Renes, Craig Costello, and Lejla Batina. *Complete addition formulas for prime order elliptic curves*. EUROCRYPT 2016. IACR ePrint 2015/1060. <https://eprint.iacr.org/2015/1060>

- [3] Daniel J. Bernstein and Tanja Lange. *Explicit-Formulas Database*. <https://hyperelliptic.org/EFD/>
- [4] Peter L. Montgomery. *Modular multiplication without trial division*. Mathematics of Computation, 44(170):519–521, 1985.
- [5] Google Security Team. *Project Wycheproof: Testing crypto libraries against known attacks*. <https://github.com/C2SP/wycheproof>
- [6] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996. <https://cacr.uwaterloo.ca/hac/>
- [7] Ernst G. Straus. *Addition chains of vectors (Problem 5125)*. American Mathematical Monthly, 71(7):806–808, 1964.
- [8] Agner Fog. *Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns*. https://www.agner.org/optimize/instruction_tables.pdf